

DESIGN SPEC DOCUMENT

ECE-593: Fundamentals of Pre-Silicon Validation
Maseeh College of Engineering and Computer Science
Winter, 2025



Project Name: Design and Verification of PCIe TL using UVM

Members: Vamsidhar Reddy Eraganeni

Saketh Kokkanti

Durgaprasad Allada

Date:

Project Name	Design and Verification of PCIe TL using UVM
Location	Portland, OR, USA
Start Date	14 th Jan 2025
Estimated Finish Date	4 th Mar 2025
Completed Date	

Prepared by: Team Number	
Prepared for: Prof. Venkatesh Patil	
Team Member Name	Email
Vamsidhar Reddy Eraganeni	
Saketh Kokanti	
Durgaprasad Allada	

Design Features:
Reset
Register Programming
Link Training
VC0 Flow Control Initialization
VC1 – VC7 Flow Control Initialization
Enumeration
Memory Write
Memory Read
Configuration Write
Configuration Read
IO Write
IO Read
Message Passing
Error

Project Description:

Modern electronic devices are demanding for high bandwidth. This can be achieved by (Peripheral Component Interconnect Express) PCI Express. PCI Express uses packet-based communication, packets can be generated in transaction layer. PCI Express is used where the high speed is required for the transfer of data. Transaction Layer supports different types of transactions such as Posted and Non-Posted transactions. PCI Express uses the serial interface through which clock skews can be reduced and higher throughput can be achieved which is useful in chip verification because chip verification depends on throughput rate. Universal Verification Methodology (UVM) is used for the verification of Transaction Layer Packet. In this, verification environment is created using four Universal Verification Component such as Advanced Peripheral Bus (APB) UVC, Advanced eXtensible Interface (AXI) UVC, Data Link Layer Transmit (DLL_TX) UVC and Data Link Layer Receive (DLL_RX) UVC. These UVC components are going to help to drive the transactions. Design Under Test (DUT) consists of Transaction Layer Packet. Constrained random verification used for complex system. In this paper the verification is done for the PCI Express Transaction Layer to verify the transactions. Simulation results are shown in Questasim tool

Important Signals/Flags

parameter S_IDLE	= 5'b0_0000;
parameter S_REG_WRITE	= 5'b0_0001;
parameter S_REG_READ	= 5'b0_0010;
parameter S_LINK_TRAINING	= 5'b0_0011;
parameter S_VC0_FC_INIT	= 5'b0_0100;
parameter S_VC1_FC_INIT	= 5'b0_0101;
parameter S_VC2_FC_INIT	= 5'b0_0110;
parameter S_VC3_FC_INIT	= 5'b0_0111;
parameter S_VC4_FC_INIT	= 5'b0_1000;
parameter S_VC5_FC_INIT	= 5'b0_1001;

```

parameter S_VC6_FC_INIT          = 5'b0_1010;
parameter S_VC7_FC_INIT          = 5'b0_1011;
parameter S_ENUMERATION          = 5'b0_1100;
parameter S_ENUM_COMPLETE_IDLE = 5'b0_1101;
parameter S_MEM_WR               = 5'b0_1110;
parameter S_MEM_RD               = 5'b0_1111;
parameter S_CFG_WR               = 5'b0_0000;
parameter S_CFG_RD               = 5'b1_0001;
parameter S_IO_WD                = 5'b1_0010;
parameter S_IO_RD                = 5'b1_0011;
parameter S_MSG                  = 5'b1_0100;
parameter S_ERROR                = 5'b1_0101;

```

Design Signals

// Processor Interface Signals

```

input aclk, arst, awvalid_p, wvalid_p;
output reg awready_p, wready_p;
input [3:0] awid_p; // This is used for transaction ID
input [31:0] awaddr_p;
input [3:0] awlen_p;
input [1:0] awburst_p;
input [31:0] wdata_p;
input [3:0] wstrb_p;
input [3:0] wid_p;
input wlast_p;
output reg bvalid_p;
input bready_p;
output reg [3:0] bid_p;
output reg [1:0] bresp_p;

input arvalid_p;
output reg arready_p;
input [3:0] arid_p; // This is used for transaction ID
input [31:0] araddr_p;

```

```
input [3:0] arlen_p;
input [1:0] arburst_p;
output reg rvalid_p;
input rready_p;
output reg [31:0] rdata_p;
output reg [3:0] rid_p;
output reg rlast_p;
output reg [1:0] rresp_p;
```

// Memory Interface Signals

```
output reg awvalid_m, wvalid_m;
input awready_m, wready_m;
output reg [3:0] awid_m; // This is used for transaction ID
output reg [31:0] awaddr_m;
output reg [3:0] awlen_m;
output reg [1:0] awburst_m;
output reg [31:0] wdata_m;
output reg [3:0] wstrb_m;
output reg [3:0] wid_m;
output reg wlast_m;
input bvalid_m;
output reg bready_m;
input [3:0] bid_m;
input [1:0] bresp_m;

output reg arvalid_m;
input arready_m;
output reg [3:0] arid_m; // This is used for transaction ID
output reg [31:0] araddr_m;
output reg [3:0] arlen_m;
output reg [1:0] arburst_m;
input rvalid_m;
output reg rready_m;
input [31:0] rdata_m;
input [3:0] rid_m;
input rlast_m;
```

```
input [1:0] rresp_m;
```

```
// DLL_TX
```

```
input tl_dll_clk;
```

```
output reg [31:0] tx_data_o;
```

```
output reg tx_valid_o;
```

```
input tx_ready_i;
```

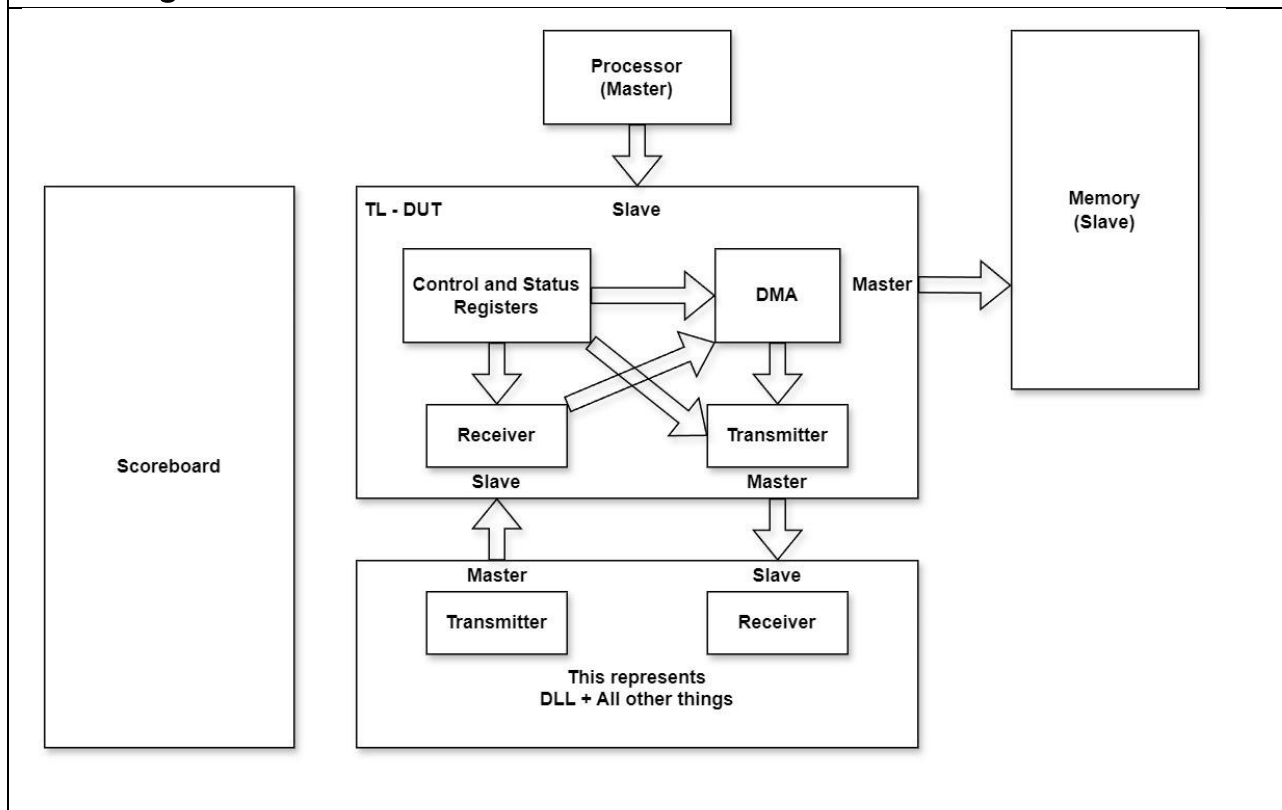
```
// DLL_RX
```

```
input [31:0] rx_data_i;
```

```
input rx_valid_i;
```

```
output reg rx_ready_o;
```

Block Diagram



References/Citations

UVM User Guide:

https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf

UVM Class Reference Manual:

https://www.accellera.org/images/downloads/standards/uvm/UVM_Class_Reference_Manual_1.2.pdf

UVM Class Library Code: [UVM-1.2](#)

PCIe 5.0 Base Specification:

<https://picture.iczhiku.com/resource/eetop/SYkDTqhOLhpUTnMx.pdf>